

CHAPTER 1 REVIEW OF RELATED LITERATURE

1.1 Generative Adversarial Networks

Generative Adversarial Networks (GANs) is a novel framework for training generative models via an adversarial process which was first introduced by ?. In the paper, "Generative Adversarial Nets", a generator network G produces samples aimed at mimicking the data distribution, while the discriminator D estimates the probability of whether an input is real or generated (?). In other words, GAN framework basically corresponds to a minimax two-player game where, a generator (G) creates samples that are intended to come from the same distribution as the training data, while the discriminator D examines the probability of the the samples provided by G being real or fake (?).

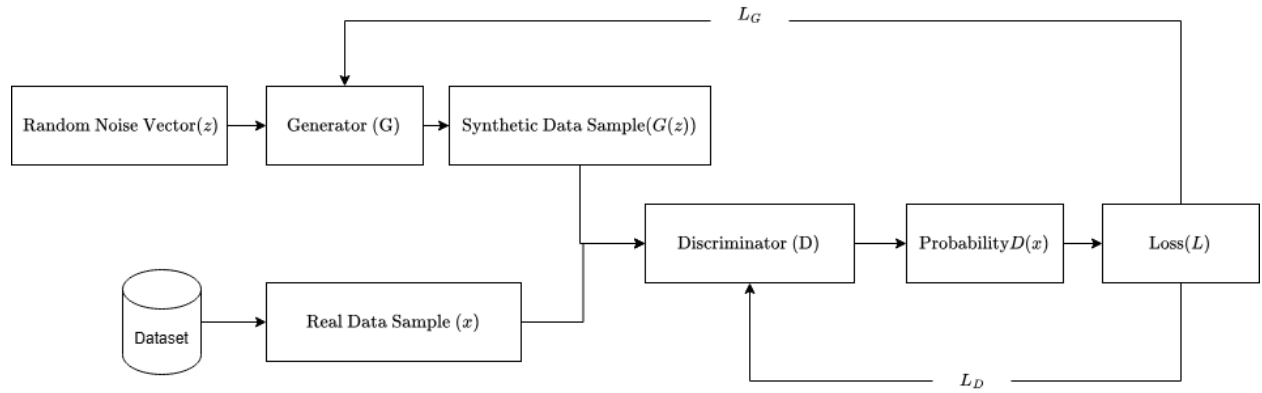


Figure 1.1 Generative Adversarial Networks (GANs) Architecture

As shown in Figure 1.1 the generator G takes as input a random noise vector $z \sim p_z$ (usually drawn from a simple distribution such as uniform or Gaussian) and outputs a synthetic data sample $G(z)$. The goal of G is to map the "latent" noise distribution into the data sample $G(z)$ is as realistic as possible. The discriminator D takes a data sample x as input and outputs a probability $D(x)$ estimating the likelihood that x is a real sample rather than one produced by G (???).

The adversarial training process in GANs is governed by a minimax optimization. Formally, ? define the value function:

$$V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}(x)}[\log D(x)] + \mathbb{E}_{z \sim p_z(z)}[\log(1 - D(G(z)))]$$

and train D and G in a two-player game:

$$\min_G \max_D V(D, G)$$

D and G are alternatively updated via minibatch stochastic gradient descent during the GAN training process. D is trained to maximize the probability of predicting whether an input from both training example and sample from G are real or fake, while G is trained simultaneously to minimize $\log(1 - D(G(z)))$ (?). In reality, GAN training can be delicate: if G becomes too strong or D too weak, gradients can vanish, and if D is too strong early on, G may receive no learning signal (saturating loss). ? suggest a non-saturating heuristic for G (maximizing $\log D(G(z))$) instead of minimizing $\log(1 - D(G(z)))$ to provide stronger gradients in practice.

On its first introduction, GANs have brought about advantages compared to other generative models at the time (e.g. Variational Auto-encoders(VAE)) like sharper images, configurable sizes, richer search space, and a veratility as it can support different generator networks. However it also has its disadvantages like : mode collapse, vanishing gradients, and instability (?).

In the years following the initial discovery of Generative Adversarial Networks (GANs), there have been numerous improvements made to their architecture that have resulted in better image quality, more stable training, and expanded capabilities. Early research highlighted the importance of convolutional structures for successful GAN performance. One influential contribution was made by ? , who introduced the Deep Convolutional GAN (DCGAN) and a set of practical design recommendations for building effective convolutional GANs. Instead

of employing traditional pooling or unpooling layers, DCGAN utilized strided convolutions to decrease spatial resolution and transposed convolutions to increase it. This allowed both G and D to learn spatial features more effectively and led to outputs that were more realistic (?). They also recommended batch normalization in almost all layers of both G and D , excluding the output and input layers respectively, which greatly stabilized training and facilitated learning proper scales.

These guidelines enabled DCGANs to synthesize high-resolution images in “one shot” and learn disentangled latent representations that enable simple vector arithmetic for changing semantic attributes of the output. DCGAN also popularized the use of Adam as an optimizer for GANs (?). In conclusion, this research provides a comprehensive overview of how convolutional structures can improve the performance of GANs, and how practical design recommendations such as strided convolutions, transposed convolutions, batch normalization, and Adam can enable these improvements. By following these guidelines, it is possible to build high-quality image synthesis models that learn complex spatial relationships in images and produce realistic outputs. In particular, DCGANs are able to synthesize high-resolution images in “one shot” and learn disentangled latent representations that allow for simple vector arithmetic for changing semantic attributes in the output, as well as enable more flexible model configurations by allowing the use of a different optimizer, such as Adam, which has been shown to be effective in training GANs. Furthermore, DCGANs popularized the use of batch normalization across all layers in both the generator and discriminator networks, excluding the output and input layers respectively, which greatly stabilized training and facilitated learning proper scales. These findings highlight the importance of convolutional structures in improving the performance of GANs and provide a comprehensive overview of how practical design recommendations can be used to build high-quality image synthesis models that learn complex spatial relationships in images and produce realistic outputs.

While StyleGAN and StyleGAN2 have achieved impressive results for image realism, they still face some challenges. The main limitation of StyleGAN lies in its inability to accurately

imitate real-world images and in the fact that it cannot directly manipulate the latent space within the network. Additionally, even when inversion is implemented, the resulting images may not always capture identity or be structurally accurate. Despite these limitations, StyleGAN has shown promise in some unstructured scenes and faces. However, as ? observed, StyleGAN struggles with domains that do not exhibit strong structure and may even replicate demographic biases in the training set. Finally, while StyleGAN can generate images at high resolution with ease, it requires extensive GPU resources to train on high-resolution data. Moreover, careful tuning of the architecture is also necessary to ensure fairness and accuracy. Overall, while StyleGAN has achieved significant progress for image realism, there are still challenges that need to be addressed in order to fully utilize its potential for creative and sophisticated use cases.

1.2 StyleGAN and StyleGAN2

Early GANs suffered from instability and mode collapse, which ? addressed by Progressive GANs (PGGAN): training begins at low resolution and gradually adds layers as resolution increases (?). Building on this, StyleGAN ? proposed a novel style-based generator architecture that achieved state-of-the-art image quality and disentangled latent factors. StyleGAN is a variant of GAN that builds on its framework by radically redesigning the generator.

Instead of feeding the latent code $\mathbf{z}$ directly into the generator, StyleGAN first maps $\mathbf{z} \sim \mathcal{N}(0, I)$ to an intermediate latent $\mathbf{w} = f(\mathbf{z})$ using an 8-layer fully-connected network (?). This $\mathbf{w}$ lives in a new “style” space $\mathcal{W}$ that is learned to be less entangled than the original latent space. Learned affine transforms then convert $\mathbf{w}$ into per-layer style parameters (y_s, y_b) that modulate each convolution via Adaptive Instance Normalization (AdaIN) (??). Concretely, for each feature map x_i in a convolutional layer, StyleGAN applies:

$$\text{AdaIN}(x_i, y) = y_{s,i} \left(\frac{x_i - \mu(x_i)}{\sigma(x_i)} \right) + y_{b,i}$$

, where y_s and y_b (scalars for channel i) are derived from $\mathbf{w}$. In practice, StyleGAN's mapping network f (an eight-layer MLP) produces $\mathbf{w}$, and then each convolutional layer is modulated by the corresponding (y_s, y_b) pair. This allows scale-specific control of features: coarse spatial styles (e.g. pose or overall shape) are set by early layers, while fine-grained details (e.g. color, texture) are set by later layers. Notably, the mapping network and style modulation enable style-mixing: two latent codes can be injected at different depths, blending coarse features from one code with fine details from another.

Importantly, StyleGAN replaces the standard random input with a learned constant $4 \times 4 \times 512$ tensor, and all variation comes from the style vectors and added noise. At each convolutional layer, StyleGAN also injects a random noise map (one per pixel) which is added to the features with learned per-channel weights. This stochastic noise produces high-frequency variation (e.g. hair wisps, freckles) without affecting global attributes. Together, these design choices yield unsupervised disentanglement of high-level attributes from stochastic detail: observe that their style-based generator “leads to an automatically learned, unsupervised separation of high-level attributes (e.g., pose and identity) and stochastic variation in the generated images”. In experiments, StyleGAN achieves state-of-the-art image fidelity (low Fréchet Inception Distance) and more linear, less entangled latent factors than conventional GANs.

The architectural differences from a standard GAN can be summarized as:

- Intermediate latent space and mapping network: StyleGAN learns a mapping $f : \mathbf{z} \rightarrow \mathbf{w}$ (eight-layer MLP) so that $\mathbf{w}$ in space $\mathcal{W}$ is disentangled
- Style modulation via AdaIN: Style vectors from $\mathbf{w}$ provide per-layer scale/bias that modulate each convolution by AdaIN, as in the formula above.
- Noise injection: Learned Gaussian noise is added to each layer to control stochastic details
- Learned constant input: The generator begins from a fixed learned 4×4 tensor instead of random noise

- Progressive growing: StyleGAN adopts the progressive training strategy from ProGAN [Karras et al., 2018], starting at 4x4 resolution and gradually adding layers to reach high resolution (?).

StyleGAN2 ? builds directly on StyleGAN but redesigns the generator to eliminate artifacts and boost quality. The key insight is that the original AdaIN normalization can destroy meaningful signal: normalizing each feature map breaks global correlations needed for coherent details. Therefore, StyleGAN2 “bakes” the style modulation into the convolution weights and replaces the per-feature normalization with a weight demodulation operation. In practice, this works as follows: let w_{ijk} be the convolution weight connecting input channel i to output channel j (at spatial position k). Given a style scale s_i (from $\mathbf{w}$) for channel i , StyleGAN2 first multiplies w_{ijk} by s_i (modulation), yielding $w'_{ijk} = s_i w_{ijk}$. It then computes the output standard deviation σ_j of the convolution and demodulates by dividing by σ_j :

$$w''_{i,s,j,k} = \frac{w'_{ijk}}{\sqrt{\sum_{i'} (w'_{ijk})^2 + \epsilon}}$$

. This eliminates the need for explicit instance normalization after the convolution. The result is that each convolutional layer can apply the style s_i directly to its weights, with the demodulation automatically preserving unit variance. This modification removes the “blob” artifacts common in StyleGAN outputs and leads to crisper, more natural images.

Training in StyleGAN2 takes a different path. Instead of slowly increasing complexity, the model uses a set structure right away. Skip connections and residual blocks link features across scales, much like in MSG-GAN or U-Net designs. This setup keeps signal flow strong throughout training. Without needing to add layers over time, stability improves naturally. The approach skips the extra burden of gradual expansion. Yet, key elements remain - style mapping and noise inputs stay part of the process. These are simply embedded into the updated generator framework. Stability meets continuity here. Not every change means replacing what worked. Another finding from ? shows that applying path length regularization to the mapping network

helps balance how shifts in W affect visual output. Because of this adjustment, movements through latent space translate more consistently into noticeable image differences. The result is a smoother connection between codes and images, easier reversal from image back to code, better separation of features, along with improved editing control.

The architectural improvements in StyleGAN2 can be summarized as:

- Weight demodulation instead of AdaIN: Styles modulate convolution weights and are normalized in-weight, removing all explicit feature-map normalization.
- No instance normalization layers: By design, the generator no longer uses AdaIN or other instance/batch norms; all normalization is implicit in weight demodulation
- Elimination of progressive growing: The network is trained at fixed full resolution, using skip/residual connections to preserve stability
- Path length regularization: A regularizer on the mapping network encourages a smooth, linearized latent space.

These changes yield a substantial quality gain. For example, [?](#) report that fully updated StyleGAN2 (no progressive growing, weight demodulation, path regularization) achieves a Fréchet Inception Distance of about 3.3 on FFHQ, compared to 4.4 for the original StyleGAN baseline.

Though StyleGAN and its successor deliver remarkable visual fidelity, certain constraints remain evident. Generated outputs along with manipulation options depend strictly on what the model learned during training. Control through latent space works solely within the boundaries of synthetic imagery the system supports; actual photographs require conversion into domain via GAN inversion prior to adjustments ([?](#)). Yet perfect preservation of subject traits isn't always achieved post-inversion. What makes face images easier for these models becomes a hurdle elsewhere - disorganized settings confuse their output. Bermano and team noted back in 2022 how StyleGAN falters when patterns lack clear repetition. Problems

show up beyond performance too, since learned behaviors include social imbalances present in source data. Training runs expose another layer of difficulty: high-res workloads need powerful graphics processors, precise adjustments. Complexity piles up, even if results look effortless.

1.3 The Latent Space

Early GANs drew a latent vector z from a simple distribution and fed it through a generator, but the resulting latent space was largely entangled and hard to control. InfoGAN by ? introduced an information-maximizing objective to encourage disentanglement, yielding interpretable latent factors. The progressive GAN (PGGAN) of ? improved image quality via multiresolution training but still used a single latent code at the input. A major shift occurred with StyleGAN ?, which used a learned mapping network to transform the input z into an intermediate latent W space. This mapping, via 8 fully-connected layers, “unfolds” the simple z distribution into W , aligning it to the generator’s feature distribution and enabling axis-parallel edits(?). StyleGAN also injects this W code and a random per-layer noise at each convolutional layer, allowing layer-wise control of coarse-to-fine image attributes.

StyleGAN2 built on this by replacing Adaptive Instance Normalization (Ada in) with weight modulation and demodulation; it further regularized the latent mapping to smooth out the generator’s response to latent shifts (?). This made the latent-to-image mapping more uniform and significantly easier to invert, meaning real images could be projected more reliably into W . Importantly, StyleGAN2 popularized the use of an extended latent space W^+ , in which a separate 512-dim w vector is supplied to each of the 18 generator layers. In W^+ , each layer can have its own style code, greatly increasing representational power: for a 1024x1024 model, W^+ is 18x512-dim. However, W^+ vectors do not come from the learned W distribution, so naive sampling can produce realistic “face-like” images but also novel patterns that are still face-like (eg. Aliens) (?). W^+ is used mostly for reconstruction/inversion and fine edits, whereas W remains the base latent space for smooth, in-distribution sampling. Finally,

StyleGAN2’s style code w is further transformed, once per layer, into a channel-wise affine vector s in Style Space S , which directly scales convolutional feature maps (?). In StyleGAN2, each channel’s style parameter in S is a single scalar, controlling only scale (no bias). Thus StyleGAN2 introduced multiple nested latent spaces: the input Z , the mapped W , the layered W^+ , and the channel-wise S .

The StyleGAN architecture encourages disentanglement by design. ? observed that W becomes “much closer to the required feature distribution” than z , so that semantic edits align with simple directions. In StyleGAN the W space yields more disentangled representations than the original Z space, making long-range manipulations (e.g. aging) more robust (?). InterFaceGAN explicitly exploits this: it identifies linear hyperplanes in W corresponding to binary face attributes (smile, age, eyeglasses, etc.) and shows that moving w across these decision boundaries toggles the attribute. In StyleGAN’s W space, these attributes become nearly disentangled after a linear transformation, unlike in Z .

Unsupervised methods have since discovered many such directions. GANSpace applies Principal Component Analysis (PCA) to StyleGAN2 latent codes and finds that principal components correspond to high-level edits (pose, color, environment) (?). Remarkably, GANSpace shows that a single global PCA direction, when applied at specific layers (“layer-wise decomposition”), can control attributes from coarse (camera angle) to fine (facial features) without labels. Similarly, ? developed an unsupervised optimization to extract interpretable latent directions from any GAN; they report non-trivial findings (e.g. a “background removal” direction) without any supervision. Semantic Factorization (SeFa) by ? provides a complementary approach by performing a closed-form eigen-decomposition of the pretrained generator’s weight matrices, SeFa factors the W space into semantically meaningful directions (necklace on/off, lighting, etc.) without training or samples. Empirically, SeFa’s directions often match or exceed those from supervised methods (e.g. InterFaceGAN) in manipulating specific attributes.

The disentangled latent structure of StyleGAN2 enables rich image manipulation. A com-

mon paradigm is latent traversal: given a latent code w , one perturbs it along a semantic direction and observes the image change. This can be simple as linear arithmetic like how InterfaceGAN uses $z_{edit} = z + \alpha n$ for single attribute manipulation. More advanced controllers have been developed. StyleFlow (Abdal et al., 2021) models latent editing as a continuous conditional flow: it learns a bijective mapping in latent space conditioned on attribute values (age, pose, gender, etc.), enabling smooth attribute-conditioned sampling and editing (?). In StyleFlow, one can fix most aspects of the face and continuously change a specified attribute like gradually make the person older with high fidelity. Similarly, StyleCLIP leverages CLIP text embeddings to find latent directions: it either optimizes a latent to match a text prompt or trains a mapper network to translate text into a direction in W or S . This yields a flexible text-based interface for latent edits (“smile”, “pikachu”, “gothic”) without requiring explicit labeled data. (?).

For image editing tasks, one often needs to map a real photo into StyleGAN2’s latent space. GAN inversion methods seek a latent code (in W , W^+ , or even feature space F) that reproduces the input under the generator. According to ?, GAN-Inversion methods suffer from fidelity-editability trade-off where optimizing in W^+ (high capacity) yields near-perfect reconstruction, but the resulting w^+ may lie outside the “safe” distribution, making edits unpredictable. Conversely, constraining to W yields more stable edits but higher reconstruction error. Modern techniques address this: encoder-based methods like pixel2Style2pixel (pSp)(?) or encoder4Editing (e4e) (?) quickly predict an approximate W^+ code; then post-processing (latent optimization or a “pivotal tuning”) refines it. For example, pSp’s encoder directly outputs 18 layer-specific style vectors into W^+ , inverting a real face in one forward pass. Such hybrid inversion plus fine-tuning allows one to start with an interpretable latent in W/W^+ and then apply the semantic edits learned for synthetic images.

Ultimately, StyleGAN2’s latent space serves as a controllable image model. One can generate images with specified attributes by sampling z and then steering $w = M(z)$ in desired directions (?). For instance, adding a “smile” direction to w yields smiling faces.

More generally, the latent allows fine-grained tuning: by selecting w^+ values per layer, one can combine styles from different images or enforce multi-attribute conditions. StyleGAN2's latent space, especially W and S , provides a semantically meaningful coordinate system. Its evolution from earlier GAN latents (simple Z) to this multi-space structure underpins the state-of-the-art in GAN-based image editing and disentangled generation.

1.4 Attribute Editing in GAN Latent Spaces

Modern Generative Adversarial Networks (GANs) have shown that the latent codes controlling image generation often embed rich semantic structure. In particular, many high-level attributes (e.g. age, smile, pose) correspond to almost-linear directions in the latent space (?). Once a direction vector d for an attribute is found, editing is done by simple latent arithmetic $w_{edit} = w + \alpha d$ where w is the original latent code α is a scalar step size, and d is the learned attribute direction. For example, InterFaceGAN treats attributes as binary labels and trains a linear SVM in the latent space; the normal vector of the separating hyperplane becomes d (?). Style-based GANs (StyleGAN1/2) enhance this by using a mapping network to produce an intermediate latent W space that is empirically more disentangled than the input Z space (?).

There are two main approaches to finding attribute directions. Supervised methods use labeled data: for a given attribute one can train a linear classifier (e.g. on whether images have glasses) on a set of latent vectors. The classifier's weight vector (normal to the decision boundary) becomes the edit direction. InterFaceGAN demonstrated this works for many facial attributes (?). There are two main approaches to finding attribute directions. Supervised methods use labeled data: for a given attribute one can train a linear classifier (e.g. on whether images have glasses) on a set of latent vectors. The classifier's weight vector (normal to the decision boundary) becomes the edit direction. InterFaceGAN demonstrated this works for many facial attributes. Unsupervised methods do not require labels: GANSpace, for instance, collects many random latents and applies Principal Component Analysis to find dominant directions. These principal axes often correspond to pose, lighting, or object type. Other

factorization techniques, like SeFa, and style-space channel-searchers similarly find promising axes with little or no supervision (??).

Once a direction d is chosen, editing is simply walking along that direction in latent space. Concretely, after inverting or sampling a code w , one produces an edited image by generating from $w + \alpha d$ for various α . InterFaceGAN likewise shows that adding the gender vector shifts a face's apparent gender. In general, Attribute Editing Models use the same principle of finding d or Δw and setting $w_{edit} = w + \alpha d$ to change attributes in an image (???)

Importantly, these methods rely on latent disentanglement: the idea that different factors are (at least partly) separable. StyleGAN's design helps here: its intermediate latent W is empirically more disentangled than the raw Z space (?). StyleGAN2's per-channel style parameters ("StyleSpace") have been shown to control very localized attributes (e.g. hair color, background elements) with minimal interference. In contrast, earlier GANs had more entangled latents, making clean edits harder (??).

However, disentanglement is not perfect. Most discovered directions still have residual cross-attribute effects. For example, one edit vector might change both "smile" and "age" together. This entanglement is noted in the literature: many control directions affect multiple attributes and can be "non-local" (?). To mitigate this, some methods perform conditional manipulation: e.g. subtracting out the projection of one direction on another to isolate each effect. But instability remains when editing multiple attributes at once, often producing unrealistic images if not handled carefully.

Attribute editing assumes that one has latent code w to edit. To apply editing to real photos, a critical step is GAN inversions. By finding a latent code that faithfully reproduces a given real image, one can apply learned semantic manipulations, adjusting age, hairstyle, or pose, to that image (?). We find a latent code z^* whose generator output matches the target image. Once z^* is found, one can apply the w_{edit} formula to manipulate the image attributes. According to ?, there are 2 gan inversion methods. Namely, the learning-based(encoder-based) and optimization based Gan inversions.

- Optimization-based inversion solves for z^* by backpropagation: it iteratively adjusts z to minimize reconstruction loss $\|G(z) - x\|$. This often yields high-fidelity reconstructions but can be slow and may get stuck in local minima.
- Learning(Encoder)-based trains a neural network $E(\cdot)$ to map images to latent codes. A trained encoder can invert images quickly, though it must be carefully regularized to align with the pretrained generator. Hybrid approaches use an encoder for an initial guess followed by optimization refinement (?).

GAN inversion is what allows editing of real images. For instance, the survey by ? illustrates how an inverted code z^* can be altered by adding attribute vectors n_1, n_2 to change the age or smile of a real face. They note that after inversion, the same latent-space editing techniques apply “without requiring any ad-hoc supervision”. However, inversion quality has limits: early models like Progressively Growing GAN (PGGAN)(?) were hard to invert faithfully, whereas StyleGAN’s architecture yields much better inversions (?). Imperfect inversion can blur details or introduce artifacts, which in turn limits how well edits carry over to the final image.

1.4.1 Attribute Group Editing

Beyond single images, recent work uses latent editing for few-shot image generation: synthesizing new images of a novel category given only a few examples. The idea is to treat each new class as a ‘set of attributes’ and apply the learned edit directions to produce more samples. A pioneering approach is Attribute Group Editing (AGE) (?). AGE assumes that images are combinations of category-relevant and category-irrelevant attributes. It represents a category by its class embedding w_c : the mean latent of the few examples. The attributes that define the category (e.g., animal shape) are captured in w_c . AGE then learns a sparse dictionary of category-irrelevant attribute directions (e.g. fur color, pose) by doing sparse coding on the differences $w_{sample} - w_c$ for many samples. The key assumption is that an attribute direction (such as ‘smiling’ or ‘red flower’) is shared between categories. To generate new images of an unseen category, AGE starts with the class embedding w_c and adds combinations of

these dictionary directions, creating diverse variations while keeping the features of the core category fixed. This editing-based generation requires no GAN retraining and can produce high-quality, diverse images even from one or few inputs.

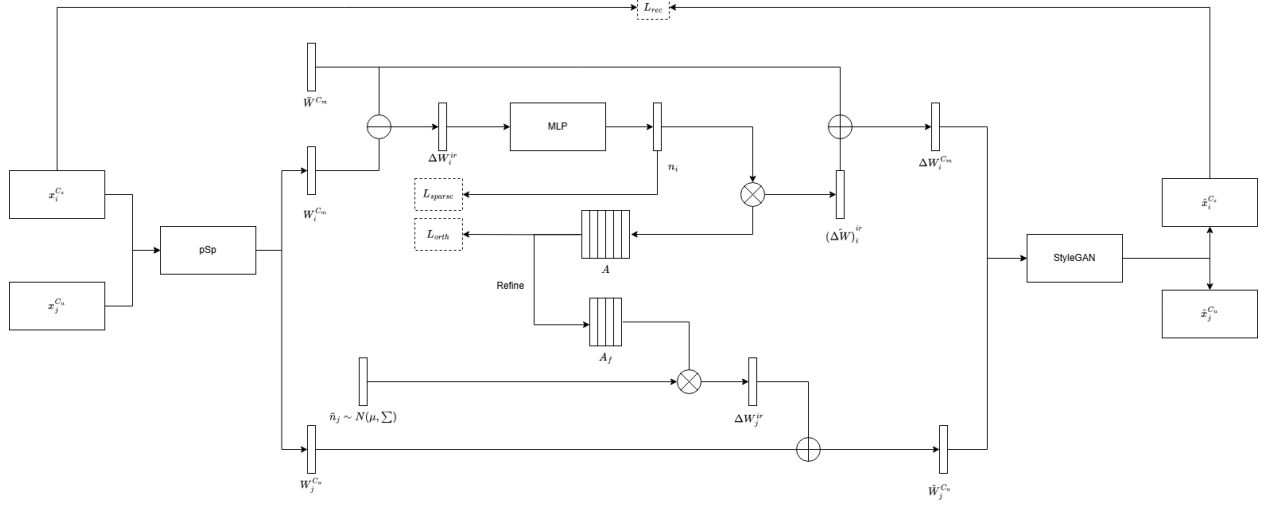


Figure 1.2 Attribute Group Editing Framework

AGE provides a powerful framework, but has some limitations. The SAGE (Stable AGE) follow-up aims to improve class consistency. Instead of editing from just one example, SAGE uses all a few-shot images to robustly estimate the class embedding. It leverages the same attribute dictionaries but selects appropriate directions based on the new class’s similarity to the seen classes. In effect, SAGE ‘injects the whole distribution of the novel class into StyleGAN’s latent space’, preserving the identity of the category. This leads to more stable outputs that help downstream tasks (e.g., classification) perform better on the generated images (?).

Most recently, TAGE (Trustworthy AGE) has extended this line further. TAGE also uses a codebook (dictionary) of attributes, but introduces transformer-style modules. Its Codebook Learning Module (CLM) finds semantic directions for category-related and unrelated attributes and builds a sparse dictionary. A Code Prediction Module (CPM) then uses global combinatorial reasoning (long-range attention) to predict the best edit codes for a given few-shot class, improving diversity and accuracy. Crucially, TAGE adds a Prompt-Driven Semantic Module

(PSM): it generates textual or semantic prompts that are injected into transformer layers, guiding fine-grained editing (?).

1.4.2 Stable Attribute Group Editing

Stable Attribute Group Editing (SAGE) was proposed as a method to overcome such limitations of AGE. Instead of editing from a single exemplar, SAGE initially employs all the given few-shot images to estimate a stable class-center embedding. It does so by capitalizing on the pretrained category-relevant attribute dictionary (learned from visible classes) to linearly represent the novel class centroid in the latent space. As described by Ding et al., since there are not a sufficient number of examples to calculate a credible mean directly, "the category-relevant dictionary... is complete enough to cover the unique attributes of any related categories," a linear combination of the dictionary atoms can "recover the class-specific features of the unseen classes" (?).

In practice, SAGE projects the few-shot samples onto such an attribute dictionary to separate their shared class features from each other, essentially removing the attributes not related to categories and retaining the attributes that are related. According to the authors, such a disentanglement also helps the model to estimate a more accurate class embedding even for a very small number of samples. The experiments also indicate that the estimated embedding from only three face images well approximates the ground truth centroid over hundreds of actual instances.

In the same paper, the weights from the projection are utilized for adaptively choosing which category-irrelevant editing directions to use. Taking cues from biological taxonomy, the authors make the assumption that categories sharing the same relevant characteristics which is being members of the same genus, for example they also share compatible and safe irrelevant characteristics. Consequently, SAGE picks the category-irrelevant directions from the highly similar seen classes depending on the projection coefficients. The approach enables the method to introduce the distribution of the unseen class to StyleGAN's latent space by

essentially repositioning the generated sample cloud towards the actual class manifold. Anchoring the synthesis about this reconstructed embedding and then carrying out filtered edits, SAGE manages to maintain class identity despite generating diverse and realistic variations.

This refinement of AGE yields clearer gains in category consistency and diversity. Quantitative results show that SAGE produces samples with significantly better quality (lower FID) and greater diversity (higher LPIPS) than AGE and competing few-shot methods. For example, without retraining the GAN, “SAGE generates images with higher quality and diversity” than prior fusion- or transformation-based approaches, and its class-embedding mapping makes it “much more stable than AGE”. Qualitatively, Ding et al. illustrate that AGE could produce bizarre artifacts (a “cat with a dog nose” or a “bird with two heads”) when editing from extreme inputs, whereas SAGE’s tailored embedding and adaptive edits yield realistic variants without such errors. In the latent space of an unseen class, AGE’s few-shot samples often lie at extreme positions and its generated points cluster narrowly around them. By contrast, SAGE’s injections form a much wider latent cloud that more closely matches the true distribution. The authors summarize that SAGE “relocates the whole distribution of the novel class in the latent space, thus effectively avoiding category-specific editing and enhancing the stability”. Overall, Ding et al. report that SAGE achieves “more stable and reliable few-shot image generation” with significantly better category retention than previous methods.

Nonetheless, SAGE does not solve all challenges. The authors note that its generative range is still more restricted than that of real data: the “dynamic range” of editable variations in SAGE is smaller than in the true class. In practice, SAGE currently uses a uniform editing intensity for all directions (since estimating per-attribute scales from few samples is infeasible) and its dictionary contains only attributes that recur across the seen classes. Consequently, some subtle class-specific variations may still be missing. As the paper observes, “SAGE simply assumes a unified intensity for editing direction sampling,” and capturing attribute-dependent editing amplitudes or class-specific edits remains an open issue. Moreover, while classification performance is improved over AGE, it still generally lags behind a model trained on real

examples. This gap – and the reliance on a pretrained StyleGAN inversion as the generator – highlight future work: extending SAGE’s ideas to handle multi-scale attributes and other generative models may be needed to fully close the gap for few-shot tasks.

1.4.3 Datasets and Applications

Attribute editing techniques are typically developed and evaluated on diverse image domains. The datasets Animal Faces HQ and Flowers are common: they have many labeled attributes and StyleGAN models are readily available for them. Few-shot generation research has also expanded to other domains. For instance, the Flowers dataset has 102 flower species (?), and Animal Faces (or AnimalFacesHQ) has 149 animal species.

Latent editing has practical use in data augmentation (few-shot generation) and interactive image manipulation. For example, one can start with a single animal or flower image and synthesize variants by editing color, pose, or environment using learned directions. However, note that many attributes do not directly map across domains; cross-domain transfer of semantic concepts remains challenging.

1.4.4 Attribute Editing limitations and challenges

Despite their promise, latent-space attribute editing methods face several challenges (?????):

- **Attribute entanglement.** Discovered directions often change multiple features at once. For instance, moving along an “age” vector might also alter “expression”. Entangled edits require careful design (e.g. subtracting projections) or more sophisticated disentanglement techniques. Fully disentangling a GAN’s latent space is still an open problem.
- **Multi-attribute interactions.** Applying more than one edit vector can be unstable. Sequential edits may accumulate artifacts, and combining directions (e.g. “add glasses” and “increase smile”) can produce unrealistic results if the attributes interact in complex ways. Ensuring consistent multi-attribute editing (for example, editing several facial features simultaneously without distortion) remains tricky.

- Inversion quality. Editing real images relies on GAN inversion. If the inversion does not perfectly capture the original, edits may fail or degrade fidelity. Early GANs (like PGGAN) struggled with inversion. While StyleGAN produces better inverted results, truly lossless inversion is hard. Any residual reconstruction error limits how cleanly attributes can be edited.
- Generalization to diverse datasets. Most latent-editing research has centered on well-studied domains (faces, common objects). Extending these methods to fine-grained categories like birds or flowers raises issues. The learned GAN (or its latent semantics) may not capture useful directions in these domains, especially if attribute labels are scarce. Additionally, attributes meaningful in one domain (e.g. “nose shape” in faces) do not translate to others. New methods must learn or adapt directions that make sense for non-face classes.

1.5 GAN Inversion

GAN inversion refers to finding a latent code in a pretrained GAN (here StyleGAN2) that reproduces a given real image when passed through the generator. In practice, one inverts an image x by solving for a style code w such that the StyleGAN2 generator G outputs an image $G(w) \approx x$ (?). This can be done via optimization (iteratively adjusting w for each image, which is slow) or via a learned encoder (a neural network that predicts w in one forward pass) (??). StyleGAN2 uses a mapping network from a base Gaussian z into an intermediate latent space W , which is more disentangled. Many inversion methods actually use an extended latent space W^+ (one 512-D style vector per layer) for greater expressiveness (???). Once an image is inverted to a latent code, one can edit the image by manipulating that code (e.g. adding learned attribute directions or swapping style layers) and generating the edited image via G . High-quality inversion thus requires balancing reconstruction fidelity (low distortion between x and $G(w)$) with editability (the code lies in a region where semantic edits are

meaningful).

How GAN inversion works (???):

- Pretrained StyleGAN2: Use a generator G trained on a target domain. Fix its weights.
- Latent spaces: Identify the latent space to invert into (typically W or extended W^+ sometimes an additional feature space F).
- Encoder or optimization: For encoder-based inversion, train a feed-forward network E to map images to latents. For optimization-based inversion, for each image x optimize W by minimizing a loss $\|x - G(w)\|$. Encoder methods are much faster (one pass) but historically had worse fidelity than optimization.
- Architecture of encoders: Typically, an encoder is a convolutional backbone (often ResNet) that produces latent parameters. For example, pixel2style2pixel (pSp) uses a ResNet + feature pyramid, producing style vectors via map2style blocks for each scale. Encoder4Editing (e4e) is similar but outputs one base style plus per-layer offsets. ReStyle applies such encoders iteratively, and StyleFeatureEditor uses a two-branch encoder for both W^+ and feature-latents.
- Loss functions: Encoders are trained to minimize a combination of losses on reconstruction. Common losses are pixel-wise L2, perceptual (LPIPS) loss, and often an identity or feature loss to preserve semantics (e.g. using ArcFace or self-supervised features). Many methods also add regularization: for example, e4e adds L2 penalties on offsets to keep the final code near W , and some use adversarial (latent discriminator) loss to keep codes within the true latent distribution.
- Inference and editing: At test time, given image x , the encoder predicts a latent code (or code+offsets or code+features). This code is fed into the fixed generator G to reconstruct x . For editing, one modifies the latent (e.g. by style mixing or adding semantic direction vectors) and uses G to synthesize the edited image.

1.5.1 pixel2style2pixel (pSp)

pSp is an early StyleGAN2 encoder (CVPR 2021) that maps an image to the extended latent space W^+ . It uses a ResNet-50 backbone with a feature pyramid producing three scales of feature maps. Each scale is fed through small map2style networks to generate 512-D style vectors corresponding to StyleGAN’s layers. The output is a full W^+ . The encoder is trained with a weighted sum of losses: pixel L2, LPIPS perceptual loss, and an identity loss (ArcFace for faces, or a self-supervised ResNet for other domains). A small regularization term on the styles can also be used. By design, pSp prioritizes reconstruction fidelity. It “directly maps a real image into the W^+ latent space with no optimization required”. In experiments it achieves low LPIPS and MSE reconstruction error. Because it predicts full W^+ , pSp can accurately capture fine details of the image. After encoding, image editing is done by standard StyleGAN manipulations (e.g. adding latent offsets or style-mixing). For example, pSp naturally supports style-mixing for multi-modal synthesis: one can replace selected style vectors with those from a random code to generate variations.

pSp is fast (single forward pass) and yields high-quality reconstructions. It has been shown to preserve identity and details (e.g. hairstyle, lighting) better than many earlier methods. It also generalizes to various image-to-image tasks without retraining (e.g. frontalization, super-resolution) because of the rich StyleGAN latent space.

However, since pSp uses full W^+ , the encoded latents can lie far from the original distribution learned in W . This can degrade “editability”: edits in regions of W^+ far from the data manifold may produce unrealistic results. pSp’s very high reconstruction fidelity sometimes comes at the cost of less coherent edits, compared to encoding into W or constrained spaces. (?)

1.5.2 Encoder4Editing (e4e)

Encoder4Editing (e4e, ICCV 2021) extends pSp by explicitly addressing the reconstruction editability trade-off. The key idea is to keep the inverted latent closer to the original W

distribution even while allowing high-fidelity reconstruction. Architecturally, e4e still uses a ResNet+feature pyramid encoder, but instead of predicting all style vectors independently, it predicts one base style code w_0 and a small offset vector o_i for each of StyleGAN’s 18 layers. The final style for layer i is $w_0 + o_i$. During training, two principles are enforced:

1. Progressive training: initially predict only w_0 then gradually allow one layer’s offset at a time, progressing from coarse to fine layers.
2. Offset regularization: penalize the magnitude of the offsets (via L2) to keep $w_0 + o_i$ close to w_0 and thus close to the learned W manifold. A small latent discriminator (GAN loss in latent space) is also trained to distinguish real W samples from the encoder’s outputs.

During training e4e uses the same image-space losses as pSp (L2, LPIPS, identity) and adds the above editability losses. The result is an encoder that strikes a balance: it produces reconstructions nearly as good as pSp, but the codes are “closer to W ” so that semantic edits transfer more reliably. e4e still outputs an extended style code, but with built-in bias towards W . In practice one can edit e4e codes using the same methods (adding latent directions, style mixing), and edits tend to be more robust. For faces, e4e also incorporates a cosine identity loss (ArcFace or general ResNet) to preserve identity under reconstruction.

e4e greatly improves editability. It was shown to maintain coherent attribute manipulations (age, expression, pose) on real images better than pSp, with only a minor increase in reconstruction error. Its inference speed is still one forward pass (plus negligible overhead from the latent discriminator at training time). It generalizes to multiple domains (faces, cars, etc.) by using a domain-agnostic feature extractor for identity.

To achieve editability, e4e accepts a small drop in reconstruction fidelity. Very fine details may be slightly lost compared to pSp. Also, the progressive training and latent GAN add complexity, and tuning the trade-off weights requires care. The notion of “proximity to W ” is somewhat loosely defined, so extremely out-of-distribution images (e.g. non-human faces) may still not be perfectly encoded. ?

1.5.3 ReStyle

ReStyle (ICCV 2021) focuses on improving inversion accuracy by iterative refinement. Instead of a single encoder pass, ReStyle’s encoder is applied multiple times in a feedback loop. Starting from an initial latent (the average w vector), ReStyle repeatedly concatenates the original image with the generator’s current reconstruction and feeds them into the encoder. At each iteration t , the encoder predicts a residual $\Delta w^{(t)}$ to add to the current latent.

$$\text{At step } t, w^{(t)} = w^{(t-1)} + E(x, G(w^{(t-1)}))$$

,

with $w^{(0)}$, set to the average latent, and $G(w)$, the current output image. The encoder is trained to minimize reconstruction loss at each step (summing L2+LPIPS over all iterations). ReStyle can be applied on top of any base encoder (e.g. pSp or e4e). In implementation, the authors actually simplify those encoders by using only the final feature map (no pyramid). Each iteration uses a fresh forward pass through the same network (weights are shared across steps). The training, therefore, unrolls a small multi-step network with losses on each reconstructed image.

ReStyle yields a more accurate latent code after its iterations, meaning $G(w)$ more closely matches the input. This generally improves any downstream editing (since the base reconstruction is better). In other words, it aims to match the quality of per-image optimization (but much faster). The final code is in W^+ and can be edited normally.

ReStyle significantly improves reconstruction quality compared to standard feed-forward encoders. It achieves near-optimization accuracy with only a small inference-time cost (typically 3-5x forward passes). Because it uses the same encoder architecture at each step, it avoids complex modifications. It can even be viewed as a learned optimization in latent space.

The iterative loop means inference is slower (several passes). Although small (e.g. 5 steps), it is no longer a single-shot encoder. Also, ReStyle does not explicitly enforce editability

constraints - it focuses on fidelity. It inherits the latent space choice of the base encoder: if initialized in W^+ the result can still lie outside the original W distribution. Finally, training is more complex (unrolling multiple steps with deep supervision) and may be sensitive to number of iterations. (?)

1.5.4 StyleFeatureEditor

StyleFeatureEditor (SFE) introduces a new latent representation combining StyleGAN's usual style codes, w or w_i , with a feature tensor F_k from an intermediate generator layer. Intuitively, the feature tensor F_k , high-dimensional spatial features, captures fine detail that W^+ alone may miss. The method has two networks: an inverter I and a feature editor H . The inverter I takes an image X and predicts both a style code $w \in W^+$ and a feature map F_k . Specifically, I first maps X to w via linear layers, and also to an intermediate feature F_{pred} . Then it fuses F_{pred} with the actual generator's feature. to produce the final inversion feature F_k .

Crucially, StyleFeatureEditor trains a separate editor network $H(F, \Delta)$ that adjusts the feature tensor for editing. Given an editing direction d in n latent space, one computes the change $\Delta = F_k(w + d) - F_k(w)$ by sampling from a pretrained encoder. Then H is trained to take the inverted feature F_k and Δ and output the edited feature $F'_k = H(F_k, \Delta)$. In inference, for a desired edit d , one computes $w' = w + d$, uses H to updated $F \Rightarrow F'_k$ and generates the edited image from w', F'_k .

StyleFeatureEditor undergoes 2 phase training:

1. Inverter I is trained on real images with standard losses (L2+LPIPS+identity, plus latent regularization and adversarial loss).
2. Editor H is trained on synthetic pairs: for each real X , a pretrained W^+ encoder produces w , an edit d is sampled, yielding, X'_E by adjusting H (inverter frozen).

StyleFeatureEditor achieves extremely high reconstruction quality and preserves detail even under editing. In experiments it produces nearly indistinguishable inverted images, and

crucially it “preserves most” of those fine details when editing. By operating directly on feature maps, it can maintain textures and fine structure that pure latent edits often lose. It is capable of editing challenging out-of-domain examples and does not rely solely on W or W^+ for content. (?)

1.6 Few-Shot Image Generation in GANs

Few-shot image generation (FSIG) extends few-shot learning to generative modeling: the goal is to synthesize novel images for a new category given only a handful of training examples. This contrasts with few-shot classification, where techniques like transfer learning have proven effective (?). In FSIG, however, naively fine-tuning a powerful generative model on few examples leads to overfitting and mode collapse. According to ?, modern GANs “tend to replicate the training samples” when adapted to only 10 target images. As a result, FSIG methods typically transfer rich prior knowledge from a large source domain and then adapt carefully to the new domain. This is often framed as a style-transfer or domain-adaptation problem: a generator pretrained on a large dataset ,like FFHQ faces, is fine-tuned on a small target set without access to the source data (??). The key challenge is preserving the source model’s diversity and high-fidelity output while learning the new style from very limited data.

Generative Adversarial Networks (GANs) have been the dominant framework for FSIG. The standard paradigm is to take a GAN (often StyleGAN or BigGAN) pretrained on a large source dataset and adapt it to the new class. Early approaches simply fine-tuned all weights on the few-shot target (TGAN) (?), but this quickly overfits (the generator memorizes the few examples). Numerous strategies have been proposed to prevent collapse and preserve diversity during adaptation. or instance, FreezeD freezes some high-resolution layers of the discriminator, ADA (adaptive data augmentation) introduces random augmentations during training, and EWC applies an Elastic Weight Consolidation penalty on important weights (?). Another approach Cross-domain Correspondence (CDC) preserves the relative distances between different source-domain samples in the target embedding space (?). In practice, Zhao et

al. (2022) found that most of these GAN-adaptation methods converge to similar image quality after many iterations – the real difference is how slowly they lose diversity during training (?). In other words, diversity degradation (mode collapse) is the main bottleneck: methods that slow down this collapse (e.g. by freezing parameters or using strong regularization) tend to perform better overall.

Building on this insight, Zhao et al. propose a contrastive learning scheme to retain diversity. Their Dual Contrastive Learning (DCL) maximizes mutual information between the source and target generators: the idea is that the same latent input z should map to semantically corresponding images before and after adaptation. Concretely, they use a contrastive loss on multi-level features of the generator and discriminator to enforce that source and target outputs remain similar in high-level content. This preserves the source generator’s rich variation in the adapted model, yielding state-of-the-art results on several datasets (?). Similarly, Hong et al. (2020) introduce DeltaGAN, which decomposes generation into a reconstruction sub-network (that computes an intra-class “delta” transformation between two samples of the same category) and a generation sub-network (that applies a sample-specific delta to a base image) (?). DeltaGAN’s adversarial “delta matching” encourages high diversity by learning many distinct transformations from the limited examples.

GAN-based FSIG methods share a transfer-learning theme: leverage a pretrained model’s knowledge and carefully regularize the adaptation. Typical training strategies include fine-tuning, parameter freezing, data augmentation, and contrastive or distance-preserving losses. Their success is usually measured by metrics like Fréchet Inception Distance (FID) or LPIPS (perceptual diversity).

Transfer learning is a commonly used training strategy in generative models that starts from a pre-trained model (GAN, VAE, or diffusion) and updates it using a few target images. Pure fine-tuning can often overfit the model, making it difficult to generalize well (?). Meta-learning involves training on simulated few-shot tasks so that it can adapt quickly. Contrastive/Distance Losses are used to enforce that the adapted generator preserves re-

relationships between different domains. Data augmentation and synthesis create pseudo-examples by replacing source images with those in the target fine-tuning process. While some GANs/VAEs may use MAML or Reptile, most often they still need per-task fine-tuning and struggle to generate sharp images (?). In contrast, meta-learned GANs/VAEs have been attempted but are usually limited by their overfitting problem (?). Meta-Learning involves training on many simulated few-shot tasks so that it can adapt quickly. Data augmentation and synthesis create pseudo-examples by replacing source images with those in the target fine-tuning process. In this way, humans can better understand how to generate realistic images based on the data they have available.

Evaluation strategies are also noteworthy. FSIG is evaluated on cross-domain tasks such as generating artistic or cartoon versions of FFHQ faces, or adapting LSUN Church to “Haunted House” style, or similar sketch-to-photo tasks. Metrics include FID for quality (when enough data exists) and LPIPS or recall for diversity. ? emphasize that with only 10 real images, FID is unstable, so they instead use intra-LPIPS (average pairwise LPIPS of generated set) to quantify diversity .

Despite these advances, few-shot image generation remains challenging. Surveys note that learning from very few or zero examples is still “a serious challenge” (?). Key open issues include: catastrophic forgetting (the model loses the source distribution entirely), mode collapse (outputs lack diversity), and evaluation (how to measure quality/diversity with tiny real sample sets). Overfitting to the few examples is ubiquitous: even with strong regularization, generators tend to memorize salient features. Furthermore, different generative families have trade-offs – GANs are fast and high-quality but brittle, VAEs are stable but blurrier, and diffusion models are powerful but computationally heavy. A brief comparison: GANs excel at sharp images but require careful balancing of losses; VAEs naturally encourage covering the whole distribution (which aids diversity) at the cost of some sharpness; diffusion models sample iteratively and often capture fine detail, but naively adapting them can be even more parameter-sensitive than GANs (?).